

Functions – Simple Hinglish Summary

Source: <https://javascript.info/function-basics>

Main baat

Aksar ek jaisa kaam script me **kai jagah** karna padta hai (jaise login aur logout par message dikhana). **Functions program ke main "building blocks" hain**, unse code **bina repeat kiye baar baar** chalaya ja sakta hai.

Built-in functions hum dekh chuke hain: `alert(message)`, `prompt(message, default)`, `confirm(question)`. Apne functions bhi bana sakte hain.

1. Function Declaration

```
javascript

function showMessage() {
  alert( 'Hello everyone!' );
}
```

Pehle `function` keyword, phir **naam**, phir bracket me **parameters** (comma se alag, khali bhi ho sakte hain), phir curly braces me **body**.

```
javascript

function name(parameter1, parameter2, ... parameterN) {
  // body
}
```

Naam se **call** karte hain:

```
javascript

showMessage();
showMessage();
```

Isse **code duplication** bachta hai. Message badalna ho to sirf ek jagah (function me) badalna padta hai.

2. Local variables

Function ke **andar** declare kiya variable **sirf usi function me** dikhta hai.

```
javascript
```

```
function showMessage() {
  let message = "Hello, I'm JavaScript!"; // local variable
  alert( message );
}

showMessage();
alert( message ); // Error! message function ke bahar nahi dikhta
```

3. Outer variables

Function **bahar ke variable** ko access aur **modify** bhi kar sakta hai.

javascript

```
let userName = 'John';

function showMessage() {
  userName = "Bob"; // outer variable badal diya

  let message = 'Hello, ' + userName;
  alert(message);
}

alert( userName ); // John
showMessage();
alert( userName ); // Bob (function ne badal diya)
```

Shadowing: Function ke andar **same naam ka local variable** ho to wo outer wale ko **chhupa deta hai**. Outer variable tab hi use hota hai jab local na ho.

javascript

```
let userName = 'John';

function showMessage() {
  let userName = "Bob"; // local variable

  let message = 'Hello, ' + userName; // Bob
  alert(message);
}

showMessage();
alert( userName ); // John (unchanged)
```

Global variables: Kisi bhi function ke bahar declare kiye variables. Ye har function me dikhte hain (jab tak shadow na ho). Inhe **kam se kam use** karna achhi practice hai.

4. Parameters

Parameters se function ko **data pass** kar sakte hain.

```
javascript

function showMessage(from, text) { // parameters: from, text
  alert(from + ': ' + text);
}

showMessage('Ann', 'Hello!');      // Ann: Hello!
showMessage('Ann', "What's up?");  // Ann: What's up?
```

Function ko values ki **copy** milti hai. Function me badlav bahar nahi dikhta:

```
javascript

function showMessage(from, text) {
  from = '*' + from + '*';
  alert( from + ': ' + text );
}

let from = "Ann";

showMessage(from, "Hello"); // *Ann*: Hello
alert( from );              // Ann (bahar ki value same rahi)
```

Parameter vs Argument

- **Parameter:** function declaration ke bracket me likha variable (declaration time ka term)
- **Argument:** function call karte waqt pass ki gayi value (call time ka term)

5. Default values

Argument **na diya** to parameter `undefined` ho jata hai.

```
javascript

showMessage("Ann"); // "*Ann*: undefined"
```

Default value `=` se de sakte ho:

javascript

```
function showMessage(from, text = "no text given") {  
  alert( from + ": " + text );  
}  
  
showMessage("Ann"); // Ann: no text given
```

- Default tab bhi lagti hai jab `undefined` **explicitly** pass ho: `showMessage("Ann", undefined)`.
- Default **koi bhi expression** ho sakti hai (jaise function call). Wo **har baar** tab evaluate hoti hai jab argument missing ho.

javascript

```
function showMessage(from, text = anotherFunction()) {  
  // anotherFunction() sirf tab chalega jab text na diya ho  
}
```

Purane tarike (purane code me milte hain)

javascript

```
// undefined check  
if (text === undefined) {  
  text = 'no text given';  
}  
  
// || operator se  
text = text || 'no text given';
```

Baad me default dena

Function chalte waqt bhi check kar sakte ho:

javascript

```
function showMessage(text) {  
  if (text === undefined) {  
    text = 'empty message';  
  }  
  alert(text);  
}
```

?? zyada behtar hai jab **0** jaisi falsy values ko normal maanna ho:

javascript

```
function showCount(count) {  
  alert(count ?? "unknown");  
}
```

```
showCount(0);    // 0  
showCount(null); // unknown  
showCount();     // unknown
```

6. Value return karna

Function **return** se **result wapas** bhej sakta hai.

javascript

```
function sum(a, b) {  
  return a + b;  
}
```

```
let result = sum(1, 2);  
alert( result ); // 3
```

- **return** function me **kahin bhi** ho sakta hai. Wahan pahunchte hi function **ruk jata** hai.
- Ek function me **kai return** ho sakte hain.

javascript

```
function checkAge(age) {  
  if (age >= 18) {  
    return true;  
  } else {  
    return confirm('Do you have permission from your parents?');  
  }  
}
```

- **Bina value ke return** function ko turant exit karata hai:

javascript

```
function showMovie(age) {
  if ( !checkAge(age) ) {
    return;
  }

  alert( "Showing you the movie" );
}
```

- Jis function me `return` na ho ya khali `return` ho, wo `undefined` return karta hai.

javascript

```
function doNothing() { /* empty */ }
alert( doNothing() === undefined ); // true
```

`return` ke baad newline mat daalo

javascript

```
return
(some + long + expression)
```

JS `return` ke baad semicolon maan leta hai, to ye **khali return** ban jata hai. Lambi expression ho to `return` wali line se hi shuru karo (ya kam se kam opening bracket wahin lagao):

javascript

```
return (
  some + long + expression
  + or +
  whatever * f(a) + f(b)
)
```

7. Function ka naam kaise rakhein

Function ek **action** hai, isliye naam aam taur par **verb** hota hai. **Chhota** aur **saaf** hona chahiye jo batae ki function kya karta hai.

Common prefixes:

Prefix	Matlab
show...	Kuch dikhata hai
get...	Value return karta hai
calc...	Kuch calculate karta hai
create...	Kuch banata hai
check...	Check karke boolean return karta hai

javascript

```
showMessage(..)    // message dikhata hai
getAge(..)         // age return karta hai
calcSum(..)        // sum calculate karke return karta hai
createForm(..)     // form banata hai (aur aam taur par return karta hai)
checkPermission(..) // permission check karke true/false return karta hai
```

Ek function, ek action

Function ko wahi karna chahiye jo naam batata hai, usse zyada nahi.

Galat examples:

- `getAge` agar `alert` bhi dikhaye (sirf age get karni chahiye)
- `createForm` agar document me form add bhi kar de (sirf banake return kare)
- `checkPermission` agar "access granted/denied" message dikhaye (sirf check karke result de)

Do alag kaam ho to **do alag functions** banao. Dono saath chalane ho to teesra function bana lo jo dono ko call kare.

Ultra-short naam

Bahut zyada use hone wale functions ke naam kabhi kabhi bahut chhote hote hain, jaise jQuery ka `$` aur Lodash ka `_`. Ye **exceptions** hain. Aam taur par naam **chhota aur descriptive** rakho.

8. Functions == Comments

Function **chhota** hona chahiye aur **sirf ek kaam** karna chahiye. Kaam bada ho to use chhote functions me tod do.

Alag function ka naam hi ek achha comment hota hai. Dono versions dekho

`(showPrimes)`:

Pehla (label ke saath):

javascript

```
function showPrimes(n) {  
  nextPrime: for (let i = 2; i < n; i++) {  
  
    for (let j = 2; j < i; j++) {  
      if (i % j == 0) continue nextPrime;  
    }  
  
    alert( i ); // prime  
  }  
}
```

Doosra (alag `isPrime` function ke saath), zyada samajhne me aasan:

javascript

```
function showPrimes(n) {  
  
  for (let i = 2; i < n; i++) {  
    if (!isPrime(i)) continue;  
  
    alert(i); // prime  
  }  
}  
  
function isPrime(n) {  
  for (let i = 2; i < n; i++) {  
    if ( n % i == 0) return false;  
  }  
  return true;  
}
```

Doosre version me code ke tukde ki jagah **action ka naam** (`isPrime`) dikhta hai. Isse code **self-describing** ho jata hai. Function tab bhi banao jab dobara use na karna ho, kyunki wo code ko **structure** aur **readable** banata hai.

Summary

javascript

```
function name(parameters, delimited, by, comma) {  
  /* code */  
}
```

- Parameters me pass ki gayi values function ke **local variables me copy** hoti hain.
- Function **outer variables access** kar sakta hai, lekin sirf **andar se bahar ki taraf**. Bahar ka code function ke local variables nahi dekh sakta.
- Function value **return** kar sakta hai. Na kare to result `undefined` hota hai.
- Code saaf rakhne ke liye function me **local variables aur parameters** use karo, outer variables nahi. **Jo function parameters leta hai, unpar kaam karke result return karta hai, wo samajhna aasan hota hai** us function se jo bina parameters ke outer variables ko side effect ke tarah badalta hai.
- Naam **verb** se shuru karo aur `create...`, `show...`, `get...`, `check...` jaise prefixes use karo.

Practice Tasks (Answers)

1. Kya `else` zaruri hai?

javascript

```
function checkAge(age) {  
  if (age > 18) {  
    return true;  
  }  
  return confirm('Did parents allow you?');  
}
```

Koi fark nahi. Dono me `return confirm(...)` tabhi chalta hai jab `if` condition falsy ho.

2. `?` ya `||` se rewrite karo:

javascript

```
// ? ke saath  
function checkAge(age) {  
  return (age > 18) ? true : confirm('Did parents allow you?');  
}  
  
// || ke saath (sabse chhota)  
function checkAge(age) {  
  return (age > 18) || confirm('Did parents allow you?');  
}
```

3. `min(a, b)` function:

javascript

```
function min(a, b) {  
  return a < b ? a : b;  
}
```

4. `pow(x, n)` function:

javascript

```
function pow(x, n) {  
  let result = x;  
  
  for (let i = 1; i < n; i++) {  
    result *= x;  
  }  
  
  return result;  
}  
  
let x = +prompt("x?", '');  
let n = +prompt("n?", '');  
  
if (n < 1) {  
  alert(`Power ${n} is not supported, use a positive integer`);  
} else {  
  alert( pow(x, n) );  
}
```

Quick Summary

Topic	Yaad rakho
Declaration	<code>function name(params) { body }</code>
Local variable	Sirf function ke andar dikhta hai
Outer variable	Function access aur modify kar sakta hai
Parameters	Values ki copy milti hai
Default value	<code>function f(a, b = "default")</code>
Return	<code>return value;</code> , na ho to <code>undefined</code>
<code>return</code> ke baad	Newline mat daalo
Naam	Verb + prefix ((<code>get</code>), (<code>show</code>), (<code>check</code>)...)
Ek function	Ek hi kaam